

Assignment 3

name: Francesco Gnan, email: gnanf@usi.ch

Deadline: 10 Dec 2023 - 11.59pm

1 Language models with LSTM [90/100]

1.1 Data (20 points)

For this part, we recommend not to waste GPU time and to do everything on your computer. The following results are implemented in file `file.py`.

1. In lines 27-28 we use the function `pandas.read_json`, which is equivalent to `pandas.DataFrame` but optimized for files `.json`, to read the data as `pandas.DataFrame`, i.e. a dictionary-like container for series objects with labeled axes (rows and columns). We have previously downloaded the dataset directly from the provided `Kaggle` link. Thanks to the function `.info()`, as we do in line 31, we get to know that there are 209527 entries in the dataset and each one is related to 6 kind of informations (6 columns): `link`, `headline`, `category`, `short.description`, `authors`, `date`. We then select those headlines, whose category is `POLITICS`. We observe that they are exactly 35602 (lines 41-42) and we print and report here the very first three of them as an example:

```
21 Biden Says U.S. Forces Would Defend Taiwan If ...
24 'Beautiful And Sad At The Same Time': Ukraine...
30 Biden Says Queen's Death Left 'Giant Hole' For...
```

2. We want to "tokenize" each title, i.e. to create for each one of them a list of words. Taking into account the function `split_to_names` from Exercise 4 we implement the function `tokenization` in lines 50-60, where we use `.lower()` to have each word in lowercase and `.split()` to split each title depending on the white spaces between words. The latter choice influences the final performance of our model because simply using `.split()` we create list of words containing special characters, such as `"!';"`. If we cleaned our lists from these special characters our model would work better, since its sentence generation would not be limited by the use of these special characters it learns from our tokenization. Moreover, we append at the end of each list the `<EOS>` token to specify the end of the title. In the example in point 1.1.1 we see that the headlines concerning the politics category are not organized starting from index 1. For this reason in line 63 we apply `reset_index(drop=True)` to reset the indices before the tokenization. Finally, in lines 67-71 we use the `pickle` format to save and load the list of list we generated. We print and report here the first three sentences, already shown in 1.1.1, tokenized:

```
[['biden', 'says', 'u.s.', 'forces', 'would', 'defend', 'taiwan',
  'if', 'china', 'invaded', '<EOS>'],
```

```
[ ' beautiful ', ' and ', ' sad ', ' at ', ' the ', ' same ', ' time : ', '
ukrainian ', ' cultural ', ' festival ', ' takes ', ' on ', ' a ', '
deeper ', ' meaning ', ' this ', ' year ', ' <EOS>'],
[ ' biden ', ' says ', ' queen's ', ' death ', ' left ', ' 'giant', ' hole ',
' for ', ' royal ', ' family ', ' <EOS>' ]]
```

As we can see in these examples, some of the words are linked together with special characters, as we already discussed.

3. We now create a python list `all_words` so as to store all the words in the dataset. We perform this operation in lines 80-89, where we add the `<EOS>` token at the beginning and we append the `"PAD"` token at the very end. Afterwards, we use this list to create two python dictionaries to map each word in `all_words` into an integer value (`word_to_int`) and viceversa (`int_to_word`). With the print in line 90 we verify that we have exactly 33207 tokens within `all_words`. Once again in lines 96-100 we save and load the two dictionaries in `pickle` format. From line 102 to line 110 we create and fill a new dictionary called `histo` in order to report the most common 5 words that appear in all the titles concerning the `POLITICS` category. In Table 1 we show these words and their occurrences.

Word	Occurrences
to	10701
the	9618
trump	6895
of	5536
in	5249

Table 1: Most common words in `POLITICS` category

4. Taking inspiration from `Exercise 4`, in lines 121-143 we create a class called `Dataset`. It takes as input the tokenization of the titles concerning politics (`data_in_char` defined in line 65) and the dictionary `word_to_int`. The class uses an auxiliary function called `keys_to_values`. In fact, `data_in_char` is a list of lists; each list contains a list of tokenized words saved as strings. The class creates a new list of lists called `data_as_int`. The latter is a list of sequences of integers, each sequence being related to each newspaper title in the starting dataset. This titles transformation from strings to integers values enables our language models to deal with the dataset more easily.
5. For this task we use the suggested code in `Exercise 4`. First of all we define a function `collate_fn`. It collates individual samples into batches. In particular, it uses the function `nn.utils.rnn.pad_sequence` to pad sequences in each batch to the length of the longest sequence. This operation is performed to obtain uniform-length batches that are essential if our language models expect fixed-size inputs. Subsequently, in lines 156-163 a `DataLoader` is created with the specified batch size (64), using the collate function `collate_fn`. It shuffles the data (`shuffle=True`) during loading. The padding value is obtained from the `word_to_int` dictionary with the key `"PAD"`.

1.2 Model definition (20 points)

In lines 167-208 we define our model. It is quite simple: it consists of a LSTM layer and a fully connected linear layer. At the beginning we perform an Embedding, i.e. we represent words as continuous vectors in a high-dimensional space. This space is continuous and enables our model to learn meaningful relationships between data. As far as the parameters of the Embedding layers are concerned, `embedding_dim` defines the size of the vectors that will represent each word, i.e. it determines the dimensionality of the continuous vector space in which words are embedded. Larger values might learn more relationships between words, but they also increase the total number of model's parameters, resulting in a possible overfitting. `padding_idx` instead specifies the index used for padding tokens. In our case it is "PAD".

We choose not to use any dropout since during the training we observe no significative difference if we use it or not. The forward pass works as follows: the input `x`, i.e. the word indices, is passed through the embedding layer; then the embedded input and the previous cell state are passed to the LSTM layer; the LSTM output finally is given as input to the fully connected linear layer. The initial cell state is computed by the `init_state` function that initializes both the hidden and cell states of the LSTM. We initialize them to zero for simplicity and we add them to the device (CPU/GPU) in order to speed up our computations.

The main difference between a LSTM model and a RNN one is the ability to deal with long-term dependencies, because of the vanishing/exploding gradient problem. In fact, RNNs do not have an explicit memory cell, whereas LSTMs update a cell state which holds the relevant information about the past. Moreover, the `nn.LSTM` function provides the possibility to add a dropout between the LSTM layers, while `nn.RNN` doesn't.

1.3 Evaluation - part 1 (10 points)

- From line 212 to line 227 we define `random_sample_next` function, that samples the next word using `np.random.choice`. Given the vocabulary values of last element of a sequence, the latter is applied to the softmax of the top-k indexes and their values (where $k = 5$ by default). We take inspiration from Exercise 4, `sample_next` function. We add some specifications in order to move the computations on the GPU. For example, in line 213 we move the input tensor to the same device as the embedding weight.
- As far as the `sample_argmax` function is concerned, we directly apply `torch.argmax` to the vocabulary values of the last element of the sequence `last_out`: it returns the index of the maximum value in the `last_out` tensor, i.e. the word with the highest probability of being selected. This function is implemented in lines 229-238.
- Finally, the `sample` function in lines 240-262 takes a prompt as input and generate new words according to `random_sample_next` or `sample_argmax` until the token `<EOS>` is found. It uses the trained model in `.eval()` modality. It fills a list of indices whose length is the difference between a default maximum sequence length and the prompt length. Then it returns this list and its values can be converted into words thanks to the dictionary `int_to_word`.

1.4 Training (35 points)

1. In lines 266-277 we define our model's hyperparameters, and we put the model itself on the GPU. We choose the suggested values for the hyperparameters. Afterwards, from line 288 to line 353 we implement the function `train`. Within each epoch we set the model

in `.train()` mode and we insert a `for` loop that iterates through each batch of data provided by `dataloader`. Each batch consists of pairs of input and output sequences. To be sure that they are of the proper data type, in lines 306-307 we convert those input and output into long tensors using the `.long()` method. We add this two lines since without them we get some errors while running the training. In particular, it seems that some words are not correctly manipulated in the previous steps. We noticed that they are a significant small number and this issue concerns only special characters, such as `"!?,."`. The conversion to long tensors solves this problem. Subsequently we put input and output on the GPU and we initialize the gradients. At this point we use the function `init_state` defined in the class `Model`, to initialize the starting cell state. Then, in line 315 we use our model to compute a new state and a new output given the input and the previous state. In line 320 we compute the loss function comparing the old and the new output.

With `loss.backward()` we get the gradients, i.e. how much the loss would change with respect to each parameter in the model. With `optimizer.step()` instead we execute one Adam algorithm step in order to slightly change all the model parameters.

Before `optimizer.step()` we perform gradient clipping that is a technique used to avoid the exploding gradient problem. In line 326 we calculate the norm of all gradients and we scale them if this norm exceeds a threshold (`clip=1`). Doing so we prevent the gradients from becoming too large, resulting in values that are too big to be correctly handled by our machine.

Concerning what we are asked to report, within each batch we compute the loss average and we take its exponent in order to collect the so-called perplexity. Finally we use the `sample` function to generate some sentences starting from a random prompt (`seed`).

In the following we show: a plot of the loss average as a function of the epochs, Figure 1; a plot of the loss function computed at every training iteration, Figure 2; a plot with the perplexity values, Figure 3.

As requested, in Figures 1 and 2 the loss function goes below 1.5. Figure 2 is interesting because we observe that within each epoch the loss values are almost constant or slightly increasing. We also observe many small oscillations and discontinuous steps between one epoch and another one. Finally in Figure 3 we see that the perplexity gets closer to zero as long as we train our model. In the following we report three sentences generated by our model:

Epoch: 1/12, Loss: 7.0096, Perplexity: 1107.1915
loofa-faced the gop tax bill would be a good for the house

Epoch: 6/12, Loss: 1.8028, Perplexity: 6.0664
problematic who will be allowed to the death penalty

Epoch: 12/12, Loss: 0.6716, Perplexity: 1.9573
'still the trump campaign is using enough to stop attacking minorities

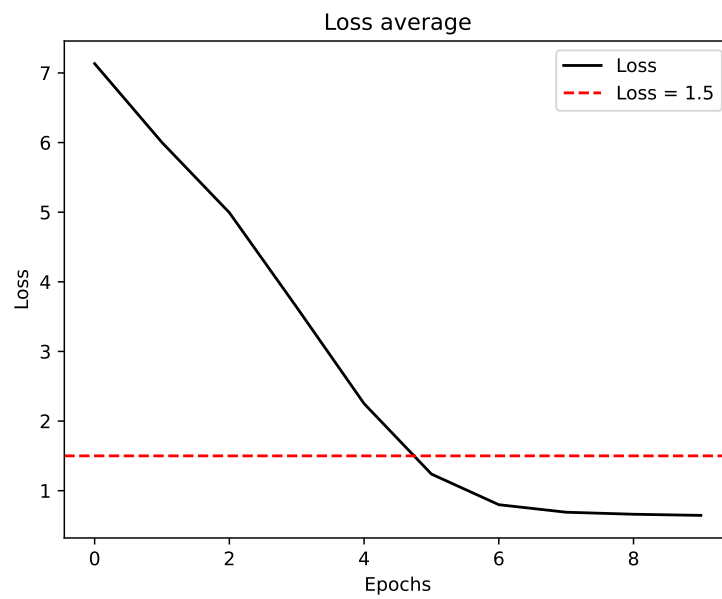


Figure 1: Loss average within each epoch as a function of the epochs

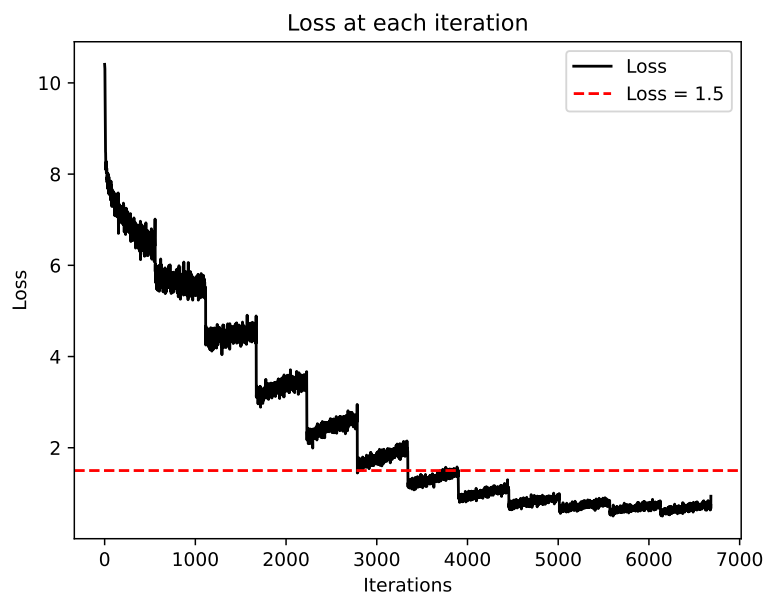


Figure 2: Loss function at each training iteration

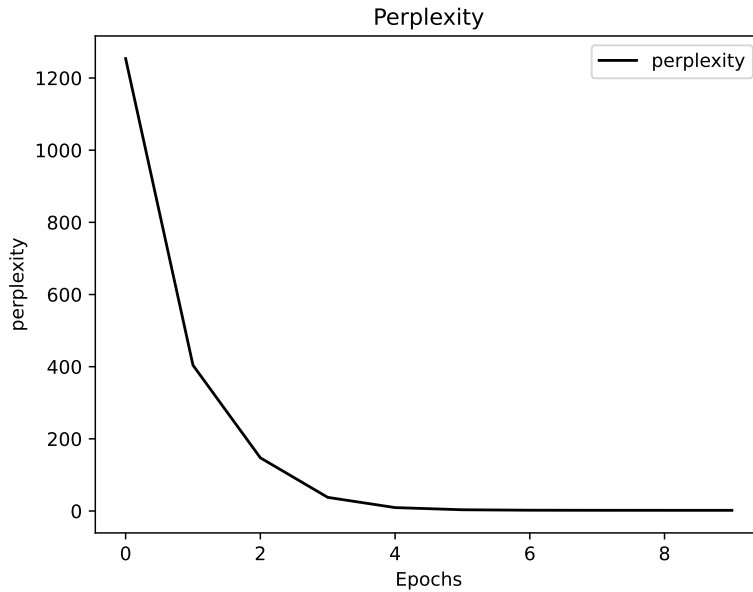


Figure 3: Perplexity as a function of the epochs

We observe once again that both the loss and the perplexity decrease through the epochs. Moreover, if we compare the second two sentences with the first one, they are more plausible.

2. In lines 406-489 we define a new function `train_with_TBBTT` which performs the truncated backpropagation through time method. In standard backpropagation through time we compute all the gradients and we update all the weights for each time step; it can lead to issues with vanishing or exploding gradients if we deal with long sequences. Differently, TBBTT breaks the sequence into smaller chunks and it updates weights based on the truncated segment of the sequence, instead of updating them based on the whole sequence. This method has advantages when addressing long sentences and in general the vanishing/exploding gradient problem. In detail, from line 425 to line 436 we build up properly the input and output chunk at each iteration and we initialize or get the hidden and cell state of the previous state. Then, in lines 441-461 we replicate what we have done in the standard previous training, but we use `input_chunk` instead of `input` and `output_chunk` instead of `output`. The main difference between `train_with_TBBTT` and `train` is that now within each batch we have a `for` loop over the number of chunks in which a given sentence is splitted. Once again we train our model using the suggested hyperparameters and we observe that, as far as the TBBTT is concerned, the training takes a bit longer. We tune the `chunk_size` parameter in order to get the requested loss/perplexity values at the end of training. We use 10 epochs instead of the suggested 5 and 2 LSTM layers, in order to get the desired final loss value. In the following we show the same kind of plots we have reported for the standard training: Figure 4 shows the loss as a function of the epochs, whereas Figure 5 shows the perplexity. We consider our model well trained since the loss reaches a value below 1 and the perplexity approaches zero.

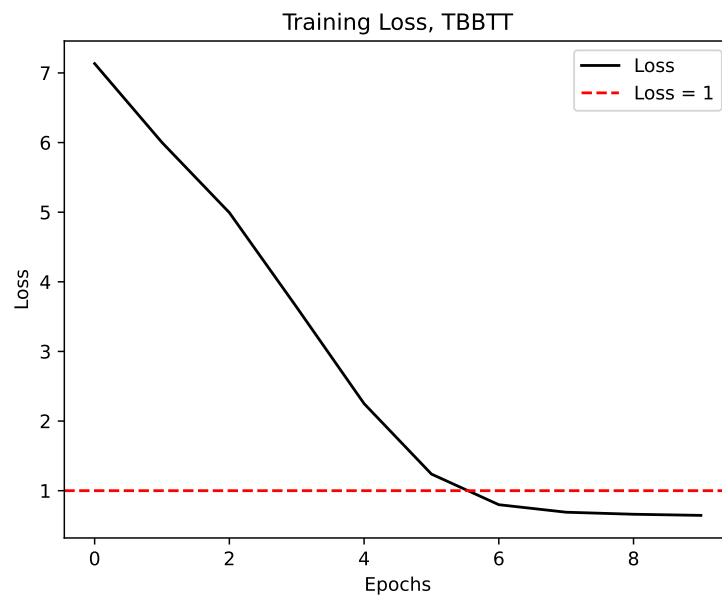


Figure 4: Loss average within each epoch as a function of the epochs

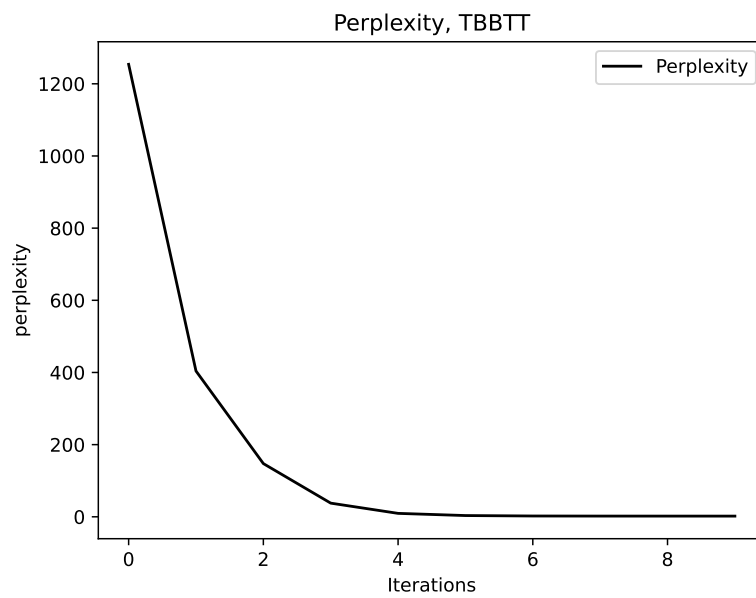


Figure 5: Perplexity as a function of the epochs

Finally, we report three generated sentences as we have done before:

```
Epoch: 1/10, Loss: 7.1341, Perplexity: 1253.9538
courtesy court says trump is a huge way to the u.s.
-----
Epoch: 5/10, Loss: 2.2507, Perplexity: 9.4943
keynote the democratic party in maryland could be a big test for
the gop?
-----
Epoch: 10/10, Loss: 0.6452, Perplexity: 1.9065
doctors the worst crisis
```

Even if the loss reaches a value that is below 1, we observe that these sentences are not plausible and they do not apply grammar rules properly.

1.5 Evaluation - part 2 (5 points)

We now generate six sentences: 3 of them using the sampling strategy, and the other 3 using the greedy one. We print them in lines 541-563.

- We observe that, as expected, the sampling method generates everytime the same sentence, which corresponds to the argmax. This happens because the selection is deterministic.

```
SAMPLING:

the president wants to know about the life of a black man
the president wants to know about the life of a black man
the president wants to know about the life of a black man
```

- Differently from the previous case, the greedy method is stochastic and each time generates a different sentence.

```
GREEDY:

the president wants a special counsel to the democratic
convention

the president wants to know about the 2018 but not the trump
court

the president wants to be more inclusive than you think
```

In general, we observe that after "wants" our model is able to predict a grammatically correct preposition. We consider some of the generated sentences credible and some not.

1.6 Bonus question* (5 points)

Given some embedding vectors related to some words in the dictionary `word_to_int`, in lines 568-603 we study if a certain mathematical operation between these vectors produces a vector whose related word is somehow close to what we expect. In particular, function `get_embedding_vector` receives a word, a model and the dictionary and returns the embedding vector associated to the word. Function `find_most_similar_words` instead gets an embedding vector, a model and a dictionary and finds the `n` most similar vectors to the given one. Using these vectors it returns the corresponding words. We compute the operation

$$\text{vector}(\text{king}) - \text{vector}(\text{man})$$

and we find among the 10 most similar words: order, leader, non-voter. In our language the latter are possible convincing candidates. To compute similarity between vectors, we use the function `cosine_similarity` that computes the angle between the two vectors. If this angle is close to 1, then the two corresponding words are likely to be similar; otherwise if it is close to -1, they are probably different.

2 Questions [5 points]

Character-level Byte-Pair Encoding tokenization technique initially constructs a vocabulary of all the unique characters in the newspaper titles; then iteratively it merges the most frequent pair of consecutive characters into a new character until a certain ending condition. The resulting vocabulary contains merged characters, which represent character-level subword units. As far as Wordpiece method is concerned, it is useful to handle rare words and out-of-vocabulary terms effectively. It starts with a vocabulary that consists of individual characters and a few special symbols, such as [CLS] (for classification tasks) and [SEP] (to separate segments of text); iteratively merges the most frequent pair of consecutive tokens in the vocabulary; the resulting vocabulary consists of subword units, and words are represented as sequences of these subwords. The vocabulary then includes both individual characters and merged subwords, allowing the model to represent a wide range of words and words variations.